

Co-pilot:

```
#include <stdio.h>
```

```
int main() {
    int secret = 7;    // fixed secret number
    int guess;
    int attempt;

    printf("Guess a number between 1 and 10.\n");

    for (attempt = 1; attempt <= 3; attempt++) {
        printf("Attempt %d/3. Enter your guess: ", attempt);
        scanf("%d", &guess);

        if (guess == secret) {
            printf("Correct! You win!\n");
            return 0; // stop immediately
        } else if (guess < secret) {
            printf("Too low! Try again.\n");
        } else {
            printf("Too high! Try again.\n");
        }
    }

    // If loop ends without a correct guess
    printf("Sorry, you lose! The number was %d.\n", secret);

    return 0;
}
```

Test execution: Run the code to verify it actually works.

- Yes, it handles sample input.

Edge cases: Check if the code handles unusual inputs, empty values, or boundary conditions rather than just the happy path.

- No, it doesn't handle a string input or any numbers outside of the range 1-10.

Error handling: Look for meaningful error messages instead of silent failures or crashing.

- Crashes if you enter a string

Time Complexity:

- uses a For loop which is the most efficient option

Space Complexity:

- used 3 constants

Maintainability:

- Limited comments.

Claude ai:

```
#include <stdio.h>
```

```
int main() {
```

```
    int secret = 7;
```

```
    int guess;
```

```
    int attempts = 3;
```

```
    int won = 0;
```

```
    printf("Guess a number between 1 and 10.\n");
```

```
    for (int i = 1; i <= attempts; i++) {
```

```
        printf("Attempt %d/%d. Enter your guess: ", i, attempts);
```

```
        scanf("%d", &guess);
```

```
        if (guess == secret) {
```

```
            printf("Correct! You win!\n");
```

```
            won = 1;
```

```
            break;
```

```
        } else if (guess < secret) {
```

```
            printf("Too low! Try again.\n");
```

```
        } else {
```

```
            printf("Too high! Try again.\n");
```

```
        }
```

```
    }
```

```
    if (!won) {
```

```
        printf("Out of attempts! The number was %d. You lose.\n", secret);  
    }  
    return 0;  
}
```

Test execution: Run the code to verify it actually works.

- Yes, it handles sample input.

Edge cases: Check if the code handles unusual inputs, empty values, or boundary conditions rather than just the happy path.

- No, it doesn't handle a string input or any numbers outside of the range 1-10.

Error handling: Look for meaningful error messages instead of silent failures or crashing.

- Crashes if you enter a string

Time Complexity:

- uses a For loop which is the most efficient option

Space Complexity:

- used 4 constants and uses an if statement after the for loop which is less efficient than either a for-else or simply a print afterword.

Maintainability:

- no comments.

Result:

I will go with the CoPilot code due to doing better or the same in every category, they both used a for loop, they both lacked error handling, CoPilot had less constants, and CoPilot had better maintainability.